

PORTADA

Análisis y Diseño de Sistemas II

Semana 6 — Modelado en UML: Diagramas de Despliegue, Paquetes y Componentes

Universidad de Antioquia · Ingeniería de Sistemas



ENCUADRE

De la conversación al diagrama formal

Ayer describieron las capas de su proyecto en tablas y conversación: presentación, lógica de negocio, persistencia. **Hoy formalizamos exactamente esa misma estructura con notación UML estándar** — tres diagramas que, juntos, documentan la arquitectura del Sprint 1.

El contenido no cambia — cambia, otra vez, la precisión con la que se comunica.

APERTURA

4 vistas
1 solo sistema



CASO · FICHA TÉCNICA

El modelo 4+1 de Kruchten, 1995

Autor	Philippe Kruchten, en Rational Software (hoy parte de IBM)
Publicación	"Architectural Blueprints — The '4+1' View Model of Software Architecture", revista <i>IEEE Software</i> , 1995
Problema que resolvía	Ningún diagrama único lograba describir un sistema completo sin volverse ilegible: un gerente no necesita ver clases; un administrador de infraestructura no necesita ver lógica interna
Solución	Cuatro vistas independientes —lógica, desarrollo, procesos, física— más los escenarios (+1) que muestran cómo trabajan juntas
Legado	Influyó directamente en la organización de UML y sigue siendo referencia estándar hoy

TRANSICIÓN

Tres de las cuatro vistas, con notación UML

UML (Unified Modeling Language) es la notación gráfica estándar, mantenida por el *Object Management Group* (OMG), para representar la estructura de un sistema. No se ejecuta — es un lenguaje para **comunicar decisiones de diseño y arquitectura entre personas**.

Hoy cubrimos tres diagramas concretos, uno por cada vista que corresponde: paquetes (lógica), componentes (desarrollo) y despliegue (física).

CONCEPTO · PANORAMA

Qué responde cada diagrama de hoy

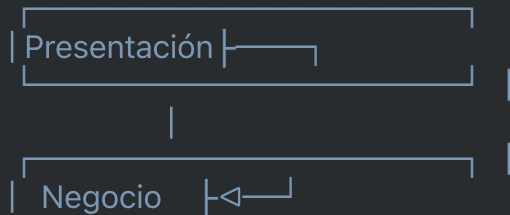
Diagrama	Vista (4+1)	Pregunta que responde
Paquetes	Lógica	¿Cómo se agrupan las clases del dominio en módulos?
Componentes	Desarrollo	¿Qué piezas construye el equipo y con qué interfaz se comunican?
Despliegue	Física	¿En qué hardware o infraestructura corre cada parte?

Los tres, juntos, formalizan las capas identificadas ayer: presentación, lógica de negocio y persistencia.

CONCEPTO · NOTACIÓN

El diagrama de paquetes

Un **paquete** se dibuja como una **carpeta** (rectángulo con una pestaña en la esquina superior izquierda) que agrupa un conjunto de clases relacionadas — normalmente las de una misma capa o un mismo módulo funcional.



CONCEPTO

Dependencias entre paquetes

Una **flecha punteada** entre dos paquetes indica que el paquete de origen **depende de** (usa clases de) el paquete de destino.

- **Presentación depende de Negocio**, y Negocio depende de Persistencia — exactamente el sentido de las capas vistas ayer.
- **Regla de buena práctica: nunca debe haber un ciclo de dependencias** entre paquetes (A depende de B, B depende de A) — es la señal más clara de que las responsabilidades no están bien separadas.



CONCEPTO · APLICADO AL PROYECTO

Paquetes, sobre las capas de ayer

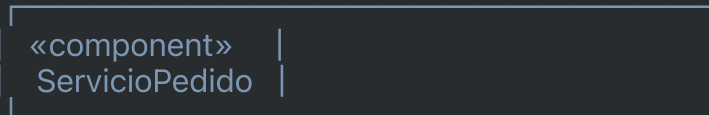
Paquete	Qué agrupa	Depende de
presentacion	Controladores, vistas, componentes de interfaz	negocio
negocio	Clases de dominio y servicios con las reglas de negocio	persistencia
persistencia	Repositorios y acceso a la base de datos	—

Este es el mismo mapeo de capas de la sesión de ayer, ahora expresado como estructura de paquetes de código.

CONCEPTO · NOTACIÓN

El diagrama de componentes

Un **componente** es una pieza de software reemplazable con una interfaz bien definida. Se dibuja como un rectángulo con el **estereotipo** «**component**» (o el ícono de dos rectángulos pequeños sobresaliendo del borde izquierdo).



CONCEPTO

Interfaz provista vs. interfaz requerida

Tipo	Notación	Significado
Provista	Círculo relleno ("lollipop") unido al componente	Lo que el componente ofrece a otros
Requerida	Media luna abierta ("socket")	Lo que el componente necesita de otro para funcionar

Dos componentes se conectan cuando el **socket** de uno encaja con el **lollipop** del otro — igual que un enchufe con su toma.

CONCEPTO · EJEMPLO

Dos componentes conectados



ControladorPedido requiere la interfaz que **ServicioPedido** provee — igual que ocurre en el código cuando el Controller invoca al Service.

CONCEPTO · DEL CÓDIGO AL DIAGRAMA

Componentes, aplicados al proyecto

Elemento del código	Elemento del diagrama de componentes
PedidoController	Componente "Controlador", expone la interfaz REST /api/pedidos
PedidoService	Componente "Servicio", provee la interfaz que requiere el Controlador
PedidoRepository	Componente "Repositorio", provee la interfaz que requiere el Servicio

Cada flecha del diagrama corresponde exactamente a una dependencia real en el código — el diagrama no es una idealización, es un reflejo fiel de la estructura.

CONTRAEJEMPLO

Mito: "UML es solo el diagrama de clases"

Es común reducir UML a un único diagrama —normalmente el de clases— porque es el más enseñado. **UML incluye más de una decena de tipos de diagrama**, cada uno útil para una vista distinta: clases y paquetes (lógica), componentes (desarrollo), secuencia y actividad (procesos), despliegue (física).

Usar solo el diagrama de clases para "documentar la arquitectura" es como describir un edificio solo con el plano eléctrico.

CONCEPTO · NOTACIÓN

El diagrama de despliegue

Un **nodo** representa hardware o infraestructura de ejecución (un servidor, un dispositivo) y se dibuja como una **caja en 3D (prisma)**. Dentro de cada nodo se colocan los **artefectos** desplegados en él (un archivo `.jar`, un contenedor).



CONCEPTO

Camino de comunicación entre nodos

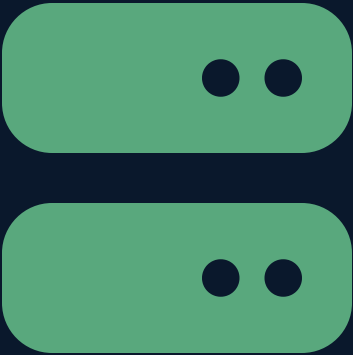
Una **línea sólida** entre dos nodos representa un **camino de comunicación** físico o de red, normalmente etiquetado con el protocolo que usa (TCP/IP , HTTPS , JDBC).

A diferencia del diagrama de componentes —que muestra dependencias lógicas entre piezas de software— el diagrama de despliegue muestra qué hardware necesita comunicarse con cuál, y por qué canal.

CONCEPTO · APLICADO AL PROYECTO

Despliegue típico de un proyecto de Fábrica Escuela

Nodo	Artefacto desplegado	Se comunica con
Navegador del cliente	Aplicación web (React/HTML/JS)	Servidor de aplicación, vía HTTPS
Servidor de aplicación	app.jar (backend Spring Boot)	Servidor de base de datos, vía JDBC
Servidor de base de datos	Motor de base de datos (PostgreSQL, MySQL)	—



CONCEPTO

Cómo se relacionan los tres diagramas

No son tres vistas aisladas: **los paquetes agrupan las clases que implementan cada componente; los componentes, a su vez, se despliegan dentro de los nodos del diagrama de despliegue.**

Presentación (paquete) → Controlador (componente) → Servidor de aplicación (nodo): la misma pieza de software, vista desde tres ángulos distintos y con tres preguntas distintas.



TENSIÓN CONCEPTUAL

¿Hay que modelar todo en UML antes de programar?

Algunos equipos insisten en documentar exhaustivamente los tres diagramas antes de escribir una sola línea de código. Otros consideran que eso retrasa la entrega sin garantizar que el modelo sea correcto — y prefieren programar primero y diagramar solo lo esencial, después.

¿Cuánto UML es "suficiente" antes de empezar a programar el Sprint 1 de este curso?



INTERACCIÓN

¿Qué diagrama responde esta pregunta?

Para cada pregunta, identifiquen si corresponde a paquetes, componentes o despliegue:

1. ¿Cuántos servidores necesita el sistema y cómo están conectados en red?
2. ¿Qué módulos existen y cómo se agrupan las clases del dominio?
3. ¿Qué componente expone la función de "registrar pedido" y con qué interfaz?
4. ¿En qué máquina corre el motor de base de datos?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. Los tres diagramas de hoy cubren tres de las cuatro vistas del modelo **4+1 de Kruchten (1995)**: paquetes (lógica), componentes (desarrollo) y despliegue (física).
2. **Paquetes**: notación de carpeta, dependencias con flecha punteada, nunca deben formar ciclos.
3. **Componentes**: estereotipo «component», con interfaces **provistas** (lollipop) y **requeridas** (socket) que se conectan entre sí.
4. **Despliegue**: nodos en 3D con artefactos dentro, unidos por caminos de comunicación etiquetados con su protocolo.
5. Los tres diagramas son la misma estructura de software, vista desde tres ángulos distintos — no vistas aisladas.



CIERRE

Para el viernes

Traigan un borrador —a mano o en la herramienta que su equipo eligió en la Unidad 2— de los **tres diagramas de hoy aplicados a su propio proyecto**.

El viernes cerramos la Unidad 3 con el Laboratorio de Modelado de Arquitectura: consolidación de estilo, capas y los tres diagramas UML del proyecto de cada equipo.

